

LLMHub Prompt Caching

Incident, Fix, and Verification Summary

Scope: `pi` coding agent · `claude-sonnet-4.6` via LLMHub · Project `Digital Application Hub Prod`

Period covered: 2026-08-14 (incident) → 2026-09-01 (fix verification)

Status: Root cause fixed and confirmed live. Two new, independently confirmed LLMHub-side bugs found during verification.

1. What happened

A `pi` session on 2026-08-14 (`cv-review` (fully anonymized content as agent input), 235 turns, ~1h51m) processed **37.39M input tokens** with ``cacheRead`/`cacheWrite`` **at 0 on every single turn**. Input per turn grew from 2,871 to 297,667 tokens, fully reprocessed each time. Session cost: **~275 €**.

2. Root cause

`pi` only attaches Anthropic's `cache_control` breakpoints when a model entry declares `compat.cacheControlFormat: "anthropic"`. This was missing from the LLMHub `claude-sonnet-4.6` entries in `~/ .pi/agent/models.json`. Not a bug in `pi`, not a gateway rejection — a client-side config gap that produced structurally valid but non-cached requests, invisible to any schema check because `cache_control` is an optional field.

LLMHub's own documentation (`docs.llmhub.t-systems.net/guides/prefix-caching`) confirms the mechanism and names a coding agent as the textbook use case — but **nowhere mentions that OpenAI-compatible clients need explicit configuration** to emit the field at all.

3. Fix applied

```
"compat": { "cacheControlFormat": "anthropic" }
```

Added to the `claude-sonnet-4.6` entry in `models.json`. Effective on next `pi` restart.

4. Verification — Test 1: real workload (cv-review, fresh key, full 16 turns)

The repo's own `docs/llmhub-cache-fix-validation-2026-09-01.md` has the complete session (16 turns, ending 12:01:13 — the copy analyzed live in this thread was cut short at turn 14). Full picture:

Turns	cacheRead	Result
1–2	0	Expected — cold start
3–11	24K–60K, climbing	Monotonic growth within one match-cv skill invocation
12, 15	reset to 24,356 / 0	Expected — each is the start of a new match-cv invocation, not a cache failure
13–14, 16	recovers immediately	Confirms resets are prefix-boundary changes, not TTL expiry or breakage

Session totals: fresh input 344,177 · output 48,668 · **cacheRead 565,825 (62.2% of input-equivalent)** · `cacheWrite` 0 · **actual cost €4.25** vs. **€7.95 hypothetical uncached** → **46.6% saved**, across 5 complete CV-match reports in ~23 minutes of active turns.

- Confirms **root cause #2** (**pi-condense** context pruning) is also resolved: `contextPrune.enabled: true` is live and fired twice this session (2 chains compressed) **without fighting the cache** — `cacheRead` grew monotonically within each pruning-eligible block rather than resetting on pruning activity, unlike the interaction flagged as a risk in the first version of this report.
- `cacheWrite` stayed `0` on all 16 turns, including both cold starts. Two explanations remain open, neither ruled out: LLMHub may not itemize cache-write billing the same way it reports `cacheRead` on this endpoint, or the backend may use GLM-style implicit/automatic caching for this path too (see §4b). Not a blocker — `cacheRead > 0` is the fix's actual success criterion, and it's unambiguous.
- Not yet exercised: a deliberate 6+ minute idle pause to force TTL expiry and observe whether `cacheWrite` finally appears. No natural gap in this session exceeded ~4 minutes.

4b. Independent confirmation at much larger scale — GLM-5.2, all projects, 2026-08-31

A separate reconciliation ([docs/cache-analysis-2026-08-31-cv-review.md](#), [docs/glm-5.2-usage-reconciliation-2026-08-31.md](#)) answered a related question raised the same day — "is a ~10M-input-token day actually a caching failure?" — across all 6 `cv-review` sessions and, separately, all projects on the machine:

- **Across all 6 Aug-31 `cv-review` sessions (562 turns, both GLM-5.2 and Claude Sonnet 5):** cache read rate **96.0%**. Only **1.1%** of ~69M "input tokens" was genuinely fresh. Estimated caching savings: **€105.23 (84.1%)**.
- **Across every project directory machine-wide for GLM-5.2 alone (50 sessions, 256 turns):** cache read rate **91.4%**; fresh input only **8.6%** of the total.
- Both reconciliations independently reproduce the same "dashboard inflates apparent usage" pattern proven manually in §7 below — at a much larger scale, and for a different model, confirming it's systemic to the dashboard's design, not specific to `claude-sonnet-4.6` or to the small-scale test in this thread.

5. Verification — Test 2: minimal, fully isolated reproduction

Two one-line prompts, then automatic `cache-warm` keep-alive pings (see §6) every 5 minutes. This produced the cleanest data of the whole investigation — small, deterministic, and it surfaced two previously unknown LLMHub bugs. → `copilot-eval-001`, fresh key

6. New finding — cache-warm runs silent background pings

The installed `cache-warm` extension (recommended and added earlier in this process) sends invisible (`display: false`) keep-alive messages roughly every 4 minutes once a session has a warm cache, to stop the 5-minute TTL from expiring. Each ping is a real, billed API call (~€0.0099). Not a bug — working as designed — but it runs unattended with no visible indicator in the terminal, and is worth a deliberate decision on whether it should be on by default.

7. Findings — LLMHub dashboard conflates cached tokens with fresh input

Proven exactly, across 8 consecutive data points in this thread's own minimal test (2-word prompts + `cache-warm` pings), matched to the token:

Dashboard checkpoint	Value shown	Matches session-log event
13:28	51,182	ping 2 ✓
13:30	64,021	ping 3 ✓
13:33	89,750	ping 5 ✓
13:36	102,640	ping 6 ✓

The dashboard's "Total input tokens" sums `fresh\_input + cacheRead` per request, instead of treating `cacheRead` as already part of the prompt (the standard OpenAI `usage` schema treats `cached_tokens` as a subset of `prompt_tokens`, not additive).

Important correction from the repo's own independent analysis: [docs/glm-5.2-usage-reconciliation-2026-08-31.md](#) checked this against the actual rendered dashboard chart and found the dashboard's own arithmetic is internally consistent — Input Tokens + Output Tokens = the Total Tokens tooltip, exactly. **This is not broken math — it's a misleading presentation.** The bar is genuinely `fresh + cacheRead + cacheWrite` summed with no visual or numeric distinction between "expensive fresh tokens" and "free/discounted cached tokens." A 9.6M-token day and a hypothetical 9.6M-all-fresh day render identically, despite one costing roughly **12x** the other. The real, billed cost (`cost.total` in the API response) correctly applies the cache discount throughout — only the token-count visualization conflates the categories.

Fix suggestion for the LLMHub team: either stack the "Input Token Usage" bar into `fresh/cacheRead/cacheWrite` segments, or annotate with a cache-hit-rate percentage per bar.

8. New finding — Dashboard reporting lag, variable, up to ~11 minutes

The dashboard was observed showing state 2–3 ping-cycles (roughly 5–11 minutes) behind the actual session log, with the lag itself fluctuating rather than growing steadily — consistent with a periodic batch job rather than a simple queue backlog.

9. Also part of this post mortem investigation

- **Public docs vs. operational portal pricing mismatch:** [docs.llmhub.t-systems.net/models/llms](#) lists `Claude 4.6 Sonnet` at €2.97/€14.85, 200K context — the actual API-key portal ([apikey.llmhub.t-systems.net](#)) confirms €7.28/€27.30, 1M context is correct. The public docs page is stale.
- **Billing-email time window error:** the automated cost-warning email said "last 30 days" while the underlying data spans ~100 days — inflating the apparent burn rate roughly 3x.
- **Original recommendation to the LLMHub team**, still standing: per-key monitoring/alerting when a large, repeated-prefix request omits `cache_control`, since the gateway already has full per-key visibility to detect this — this is what would have caught the original incident on turn one instead of after four months.

11. Bottom line

The fix works and is confirmed live in production traffic, now with two independent confirmations at different scales: this thread's minimal reproduction, and the repo's own full 16-turn `cv-review` validation plus a 562-turn, 6-session, two-model reconciliation. Both compounding root causes from the original incident — missing `cache_control` and disabled context pruning — are confirmed resolved and non-conflicting.

Four LLMHub-side platform issues are now on record, all independently reproducible:

- Public docs vs. operational portal pricing mismatch (stale docs)
- Missing `cacheWrite` reporting in the API response (mechanism unclear, not yet a blocker)
- Dashboard conflates cached and fresh tokens into one number — presentation issue, not broken math, confirmed at two different scales